

# 容器类型（列表、元组、字典、集合）

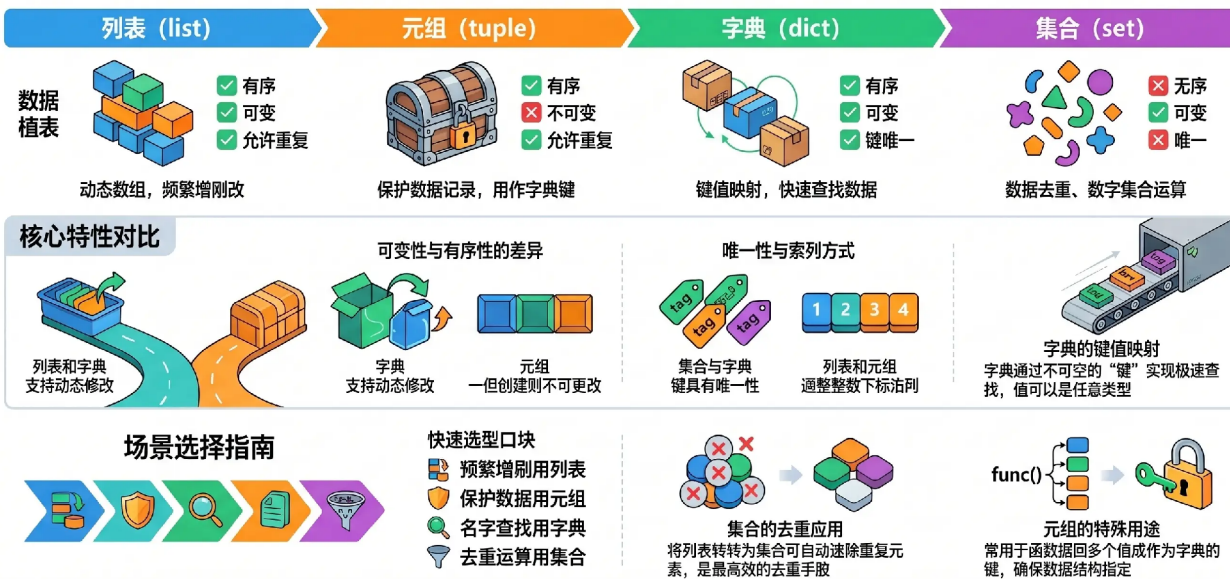


## 学习目标：

1. 理解 Python 四大容器类型（列表、元组、字典、集合）的概念与作用。
2. 掌握列表的定义、常用操作（增删改查）与遍历方法。
3. 掌握元组的特性、定义与使用场景，理解元组不可变的特点。
4. 掌握字典的键值对结构、常用方法与遍历方式。
5. 掌握集合的定义、去重特性与基本运算。
6. 能够区分四大容器的使用场景并灵活选择。
7. 能够结合循环完成容器数据的处理与应用。

## 一图看懂 Python 四大内置容器：列表、元组、字典与集合

本指南详细介绍了 Python 的四种核心数据容器：列表 (List)、元组 (Tuple)、集合 (Set) 和字典 (Dict)。通过对比它们的有序性，可变性和唯一性，为开发者清晰的选择参与和操作指南。



© NotebookLM

# 第四章：列表 (list)

## 1. 列表的定义与特性

列表 (List) 是一种 有序、可变 的数据容器，用来存储多个数据。  
它可以包含不同类型的元素 (数字、字符串、布尔值等)。

| 特性   | 说明               |
|------|------------------|
| 有序   | 每个元素有固定的索引 (下标)  |
| 可变   | 可以添加、删除、修改元素     |
| 可重复  | 允许重复的元素存在        |
| 任意类型 | 一个列表可以混合存放不同数据类型 |

## 2. 列表的创建与基本特性

python

```
1 # 空列表
2 empty = []
3
4 # 存储多个元素 (类型可以混合)
5 nums = [1, 2, 3, 4, 5]
6 names = ["张三", "李四", "王五"]
7 mixed = [10, "hello", 3.14, True]
```

## 3. 索引与切片 (与字符串类似)

python

```
1 nums = [10, 20, 30, 40, 50]
```

```

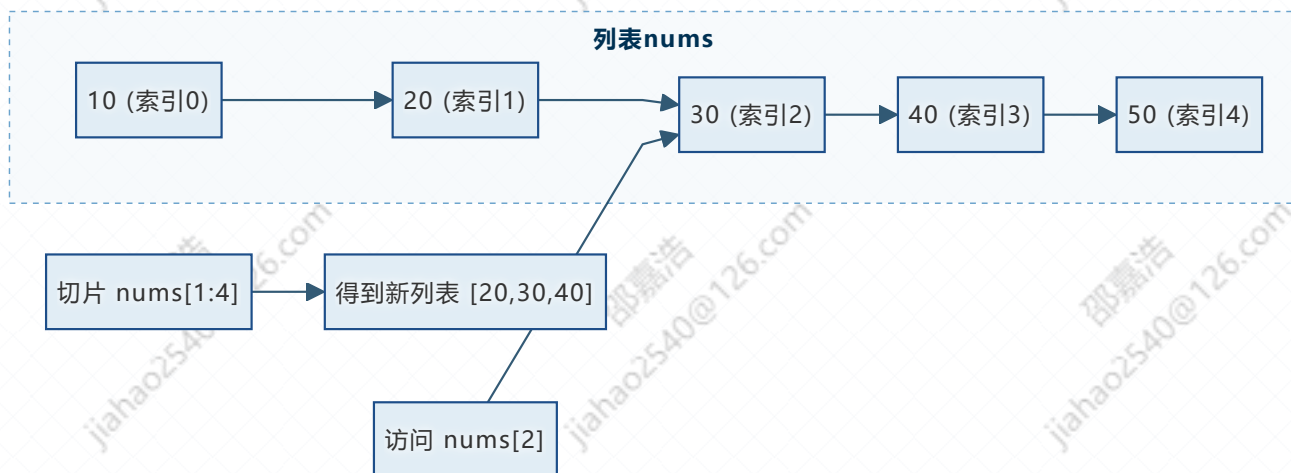
2 # 索引:  0      1      2      3      4
3 #      -5    -4     -3     -2     -1
4
5 print(nums[0])      # 10
6 print(nums[-1])     # 50
7 print(nums[1:4])    # [20, 30, 40]  注意: 结束索引不包含
8 print(nums[:2])     # [10, 30, 50]  步长为2
9 print(nums[::-1])   # [50, 40, 30, 20, 10]  反转

```

### ❗ Important

切片返回的是**新列表**，原列表不变。

### 流程图：列表索引与切片



## 4. 常用操作（增删改查）

| 操作     | 方法/语法                         | 说明                                    | 示例                               | 结果                           |
|--------|-------------------------------|---------------------------------------|----------------------------------|------------------------------|
| 追加末尾   | <code>append(x)</code>        | 在列表末尾添加一个元素                           | <code>lst.append(4)</code>       | <code>[1, 2, 3, 4]</code>    |
| 插入     | <code>insert(i, x)</code>     | 在索引 i 处插入元素 x                         | <code>lst.insert(1, 99)</code>   | <code>[1, 99, 2, 3]</code>   |
| 扩展     | <code>extend(iterable)</code> | 将另一个可迭代对象的元素追加到末尾                     | <code>lst.extend([4, 5])</code>  | <code>[1, 2, 3, 4, 5]</code> |
| 删除指定值  | <code>remove(x)</code>        | 删除第一个值为 x 的元素                         | <code>lst.remove(2)</code>       | <code>[1, 3]</code>          |
| 删除指定索引 | <code>pop(i)</code>           | 删除并返回索引 i 处的元素（默认末尾）                  | <code>lst.pop()</code>           | 返回末尾元素，列表缩短                  |
| 清空     | <code>clear()</code>          | 删除所有元素                                | <code>lst.clear()</code>         | <code>[]</code>              |
| 修改     | <code>lst[i] = new</code>     | 通过索引直接赋值                              | <code>lst[0]=100</code>          | <code>[100, 2, 3]</code>     |
| 查找索引   | <code>index(x)</code>         | 返回第一个值为 x 的索引                         | <code>lst.index(3)</code>        | <code>2</code>               |
| 统计次数   | <code>count(x)</code>         | 返回 x 在列表中出现的次数                        | <code>[1, 2, 2].count(2)</code>  | <code>2</code>               |
| 排序     | <code>sort()</code>           | 原地升序排序（可加 <code>reverse=True</code> ） | <code>[3, 1, 2].sort()</code>    | <code>[1, 2, 3]</code>       |
| 反转     | <code>reverse()</code>        | 原地反转列表                                | <code>[1, 2, 3].reverse()</code> | <code>[3, 2, 1]</code>       |
| 长度     | <code>len(lst)</code>         | 返回元素个数                                | <code>len([1, 2, 3])</code>      | <code>3</code>               |

python

```
1 # 示例演示
2 fruits = ["苹果", "香蕉"]
3 fruits.append("橙子") # ['苹果', '香蕉', '橙子']
```

```
4  fruits.insert(1, "葡萄")           # ['苹果', '葡萄', '香蕉',  
   '橙子']  
5  fruits.remove("香蕉")             # ['苹果', '葡萄', '橙子']  
6  last = fruits.pop()               # last='橙子', fruits=['苹  
   果', '葡萄']  
7  fruits[0] = "西瓜"                # ['西瓜', '葡萄']  
8  print(fruits.index("葡萄"))       # 1
```

## 5. 遍历列表

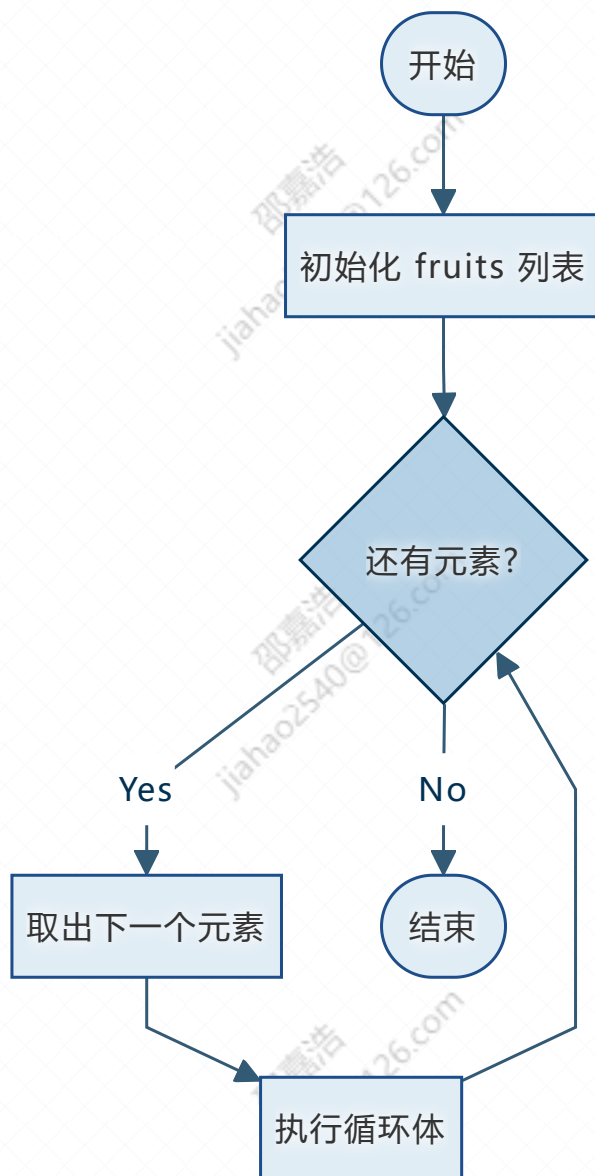


python

```
1  fruits = ["苹果", "香蕉", "橙子"]  
2  # 方式1: 直接遍历元素  
3  for fruit in fruits:  
4      print(fruit)  
5  
6  # 方式2: 同时获取索引和元素 (使用 enumerate)  
7  for idx, fruit in enumerate(fruits):  
8      print(f"第{idx+1}个水果是: {fruit}")
```

流程图：遍历列表





## 6. 列表推导式（简洁创建列表）

Python 推导式是一种独特的数据处理方式，可以从一个数据序列构建另一个新的数据序列的结构体。

Python 推导式是一种强大且简洁的语法，适用于生成列表、字典、集合和生成器。

在使用推导式时，需要注意可读性，尽量保持表达式简洁，以免影响代码的可读性和可维护性。

**语法格式：**



python

```
1 [表达式 for 变量 in 列表]
```

```
2
```

3 或者  
4  
5 [表达式 for 变量 in 列表 if 条件]

示例：

```
python
1 # 生成 0~9 的平方列表
2 squares = [x**2 for x in range(10)]
3 print(squares) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
4
5 # 带条件的推导式：筛选偶数
6 evens = [x for x in range(10) if x % 2 == 0]
7 print(evens) # [0, 2, 4, 6, 8]
```

## 7. 课堂练习

1. 创建列表 `nums = [5, 2, 9, 1, 5, 6]`，在末尾添加数字 7，删除索引 2 处的元素，对列表升序排序，输出结果。
2. 使用列表推导式生成 1~20 中所有偶数的平方，存入列表 `even_squares` 并输出。
3. 给定列表 `[1, 2, 3, 2, 4, 2, 5]`，统计数字 2 出现的次数，并输出第一个 2 的索引位置。

# 第五章：元组 (tuple)

## 1. 元组的定义与特性

**元组**是有序、**不可变**的序列，用小括号 `()` 表示。一旦创建，不能修改（增、删、改都不行）。

```
python
```

```
1 t1 = (1, 2, 3)
2 t2 = (4, 5, 6)
3 single = (5,)      # 单元素元组必须加逗号，否则是整数
4 empty = ()
```

| 特性  | 说明              |
|-----|-----------------|
| 有序  | 支持索引和切片         |
| 不可变 | 不能添加、删除、修改元素    |
| 可重复 | 允许重复元素          |
| 可哈希 | 可以作为字典的键（而列表不行） |

## 2. 索引和切片（同列表）

```
python
1 t = (10, 20, 30, 40)
2 print(t[1])      # 20
3 print(t[-2:])    # (30, 40)
```

## 3. 元组的常用操作（因不可变，方法较少）

```
python
1 t = (1, 2, 2, 3)
2 print(len(t))    # 4
3 print(t.count(2)) # 2
4 print(t.index(3)) # 3
5 print(2 in t)     # True
```



## 4. 元组的解包

**解包** (Unpacking) 是一种将可迭代对象或映射类型的元素分配到多个变量的操作，常用于数据提取、函数调用和结构化编程，可显著提升代码的简洁性与可读性。

```
python
1 point = (10, 20)
2 x, y = point          # 自动解包
3 print(x, y)           # 10 20
4
5 # 交换两个变量（利用元组解包）
6 a, b = 5, 10
7 a, b = b, a           # 交换后 a=10, b=5
```

## 5. 元组的用途

- 保护数据不被修改（例如一周的天数、坐标点）。
- 作为字典的键（因为不可变）。
- 函数返回多个值时，实际返回的是元组。

```
python
1 # 作为字典的键（列表不行）
2 locations = {(40.7128, -74.0060): "纽约"} # 合法
3 # locations = {[40.7128, -74.0060]: "纽约"} # 报错，列表不可哈希
```

## 6. 课堂练习

### 1. 元组创建与索引

创建元组 `t = (25, 30, 35, 40, 45)`，完成以下操作：

- 输出元组的第一个元素和最后一个元素（使用负索引）。
- 输出元组中索引为 2 的元素。
- 尝试执行 `t[1] = 99`，观察并解释报错信息。

### 2. 元组切片与拼接

给定元组 `a = (1, 2, 3)`，`b = (4, 5, 6)`：

- 通过切片获取 `a` 中的前两个元素，组成新元组。
- 将 `a` 和 `b` 拼接成新元组 `c`，并输出。
- 重复 `a` 两次，输出结果。

### 3. 元组解包

- 创建元组 `info = ("张三", 18, "男")`，使用解包将其中的值分别赋值给变量 `name`，`age`，`gender`，并输出 `name`。
- 利用元组解包交换两个变量的值：`x = 10`，`y = 20`，交换后输出 `x` 和 `y`。
- 编写一行代码，将元组 `(100, 200, 300)` 解包并计算三个数的和，输出结果。

## 第六章：集合 (set)

### 1. 集合的定义与特性

集合是无序、元素唯一（不重复）的可变容器，用花括号 `{}` 表示（空集合必须用 `set()`）。

```
python
1  s1 = {1, 2, 3}
2  s2 = set()           # 空集合（不能用 {}, 因为那是空字典）
```

3 | `s3 = set([1, 2, 2, 3])` # 从列表创建，自动去重 → {1, 2, 3}

| 特性    | 说明                    |
|-------|-----------------------|
| 无序    | 没有索引，不支持下标访问          |
| 唯一性   | 自动去除重复元素              |
| 可变    | 可以添加、删除元素             |
| 可哈希元素 | 元素必须是不可变类型（数字、字符串、元组） |

## 2. 集合的常用操作

```
python
1  s = {1, 2, 3}
2  s.add(4)                # {1, 2, 3, 4}
3  s.update([5, 6])        # 添加多个 {1, 2, 3, 4, 5, 6}
4  s.remove(3)             # 删除元素，不存在报错
5  s.discard(10)           # 删除元素，不存在也不报错
6  s.pop()                 # 随机删除并返回一个元素
7  s.clear()               # 清空集合
```

## 3. 集合运算（数学集合操作）

```
python
1  A = {1, 2, 3}
2  B = {3, 4, 5}
3
4  print(A | B)            # 并集 {1, 2, 3, 4, 5}
5  print(A & B)            # 交集 {3}
6  print(A - B)            # 差集（A中有B中没有）{1, 2}
7  print(A ^ B)            # 对称差（不同时属于两者的元素）{1, 2, 4, 5}
8  print(A <= B)           # 子集判断 False 判断 A 是否为 B 的子集
```

## 4 集合的典型应用：去重

python

```
1  nums = [1, 2, 2, 3, 3, 4]
2  unique = list(set(nums))    # [1, 2, 3, 4] 顺序可能打乱
3  # 如果需保持原有顺序 (Python 3.7+ 字典有序)
4  unique_ordered = list(dict.fromkeys(nums)) # [1, 2, 3, 4]
```

## 5. 课堂练习

### 1. 集合的创建与修改

- 使用 `set()` 创建一个空集合 `s1`，然后依次添加元素 `"苹果"`，`"香蕉"`，`"苹果"`（注意重复的 `"苹果"` 只应添加一次）。
- 再创建一个集合 `s2 = {"香蕉", "橙子", "葡萄"}`。
- 分别输出 `s1` 和 `s2`，观察重复元素是否被去重。
- 从 `s1` 中删除元素 `"苹果"`（若存在），删除不存在的元素 `"西瓜"` 时不报错。

## 2. 集合运算（交集、并集、差集、对称差）

给定集合：

python

```
1 | A = {1, 2, 3, 4, 5}
2 | B = {4, 5, 6, 7, 8}
```

计算并输出以下结果：

- 并集 `A | B`
- 交集 `A & B`
- 差集 `A - B`
- 对称差 `A ^ B`
- 判断 `A` 是否是 `B` 的子集

## 3. 集合去重应用

给定列表 `nums = [3, 5, 2, 3, 8, 5, 7, 2]`：

- 使用集合去除列表中的重复元素，并输出去重后的结果（不要求保持原顺序）。
- 如果保持原顺序（即第一次出现的顺序），请编写代码实现。
- 统计去重后有多少个元素。

---

# 第七章：字典（dict）

## 1. 字典的定义与特性

字典（dict）是一种 **键值对（key-value）** 的集合；

使用 **大括号{}** 定义；

**键（key）** 必须唯一且不可变（如字符串、数组、元组）；

**值（value）** 可以是任意类型。



## 语法结构：



python

```
1 字典名 = {键1: 值1, 键2: 值2, 键3: 值3, ...}
```

---

## 2. 字典的创建



python

```
1  # 空字典
2  empty = {}
3
4  # 键值对
5  person = {"name": "小明", "age": 20, "city": "北京"}
6
7  # 使用 dict() 构造函数
8  person2 = dict(name="小红", age=18)  # {'name': '小红',
    'age': 18}
```

---

### 3. 字典的访问与操作

| 操作    | 语法                               | 说明                               | 示例                                     |
|-------|----------------------------------|----------------------------------|----------------------------------------|
| 通过键取值 | <code>d[key]</code>              | 键不存在会报错<br><code>KeyError</code> | <code>person["name"]</code> → "小明"     |
| 安全取值  | <code>d.get(key, default)</code> | 键不存在返回默认值（默认 <code>None</code> ） | <code>person.get("age", 0)</code> → 20 |
| 添加/修改 | <code>d[key] = value</code>      | 键不存在则添加，存在则修改                    | <code>person["score"] = 95</code>      |
| 删除键   | <code>del d[key]</code>          | 删除键值对                            | <code>del person["city"]</code>        |
| 删除并返回 | <code>d.pop(key)</code>          | 删除并返回值                           | <code>age = person.pop("age")</code>   |
| 清空    | <code>d.clear()</code>           | 删除所有键值对                          | <code>person.clear()</code>            |
| 获取所有键 | <code>d.keys()</code>            | 返回可迭代的视图                         | <code>person.keys()</code>             |
| 获取所有值 | <code>d.values()</code>          | 返回可迭代的视图                         | <code>person.values()</code>           |
| 获取键值对 | <code>d.items()</code>           | 返回 <code>(键, 值)</code> 元组的视图     | <code>person.items()</code>            |
| 长度    | <code>len(d)</code>              | 键值对个数                            | <code>len(person)</code>               |

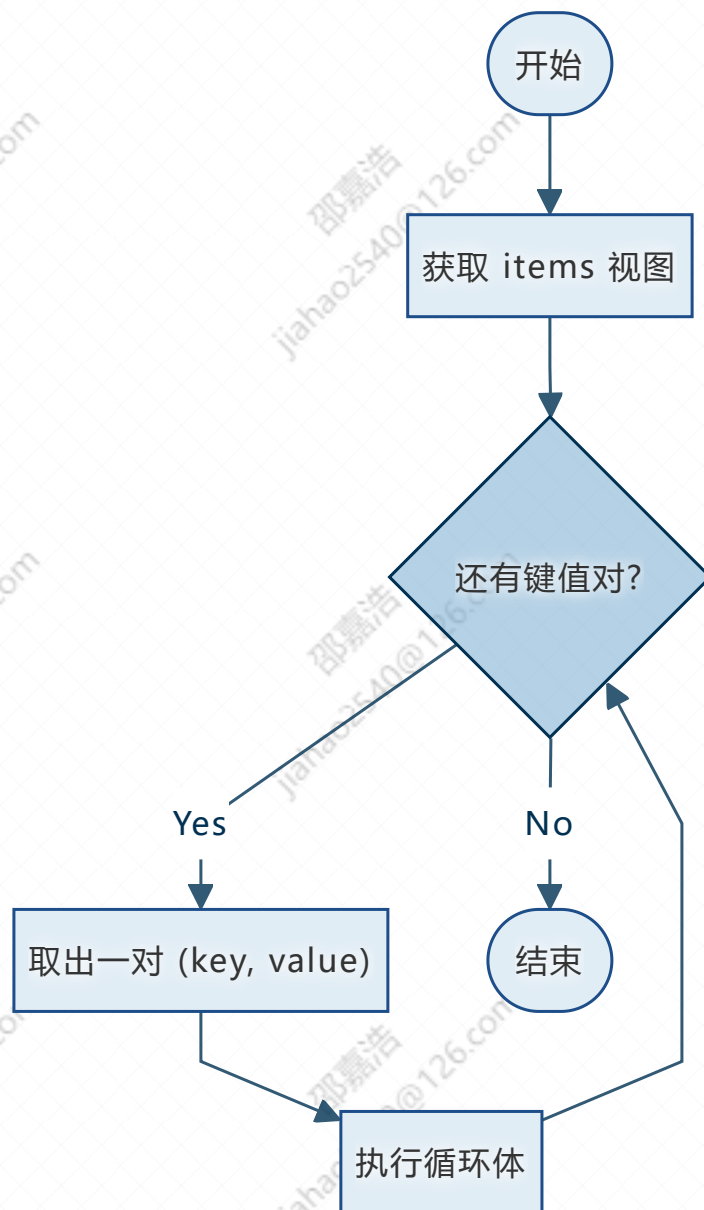
python

```
1 person = {"name": "小明", "age": 20}
2 print(person["name"])           # 小明
3 person["city"] = "上海"         # 添加
4 person["age"] = 21              # 修改
5 del person["city"]              # 删除
6 print(person.get("score", 0))   # 0（不存在返回默认值）
```

## 4. 遍历字典

python

```
1  person = {"name": "小明", "age": 20, "city": "北京"}
2
3  # 遍历键
4  for key in person.keys():
5      print(key, person[key])
6
7  # 遍历值
8  for value in person.values():
9      print(value)
10
11 # 同时遍历键和值（推荐）
12 for key, value in person.items():
13     print(f"{key} -> {value}")
```



## 5. 字典推导式

```
1 # 交换键值
2 original = {"a": 1, "b": 2}
3 swapped = {v: k for k, v in original.items()} # {1: 'a',
4         2: 'b'}
5
6 # 筛选成绩大于80的学生
7 scores = {"张三": 85, "李四": 60, "王五": 92}
```

python

```
7 passed = {name: score for name, score in scores.items() if
score >= 80}
8 print(passed) # {'张三': 85, '王五': 92}
```

## 6. 字典的常用方法

```
python
1 # 合并字典 (update)
2 d1 = {"a": 1, "b": 2}
3 d2 = {"c": 3, "d": 4}
4 d1.update(d2) # d1 变成 {'a':1, 'b':2, 'c':3, 'd':4}
5
6 # 从两个列表创建字典
7 keys = ["name", "age", "city"]
8 values = ["张三", 20, "北京"]
9 person = dict(zip(keys, values)) # {'name': '张三', 'age':
20, 'city': '北京'}
```



## 7. 课堂练习

### 1. 字典的创建与基本操作

创建一个字典 `student`，包含以下键值对：

- `"name"` : `"李华"`
- `"age"` : `19`
- `"major"` : `"计算机科学"`

完成以下操作：

- 输出 `student` 中的所有键和值。
- 使用 `get()` 方法获取 `"age"` 的值，再尝试获取不存在的 `"score"`，默认返回 `0`。
- 添加一个键值对 `"grade"` : `88`。
- 将 `"age"` 修改为 `20`。
- 删除 `"major"` 键值对。
- 清空整个字典。

### 2. 字典的遍历与推导式

给定字典 `scores = {"张三": 85, "李四": 92, "王五": 78, "赵六": 88}`：

- 使用 `for` 循环遍历字典，输出每个学生的姓名和成绩，格式如 `张三的成绩是85`。
- 仅输出所有成绩的平均分（保留一位小数）。
- 使用字典推导式，筛选出成绩大于等于 80 的学生，生成新字典 `high_scores`，并输出。

3. 字典综合应用

有一段文本: "hello world hello python world" , 请统计每个单词出现的次数, 存入字典。要求:

- 将文本按空格分割成单词列表。
- 遍历列表, 统计每个单词出现的次数。
- 最后输出单词统计结果 (例如 {'hello': 2, 'world': 2, 'python': 1} )。
- 提示: 可以使用 `split()` 方法分割字符串。

附录：容器类型对比与选择指南

| 容器            | 是否有序           | 是否可变  | 是否允许重复    | 索引方式        | 典型用途          |
|---------------|----------------|-------|-----------|-------------|---------------|
| 列表<br>(list)  | ✔ 有序           | ✔ 可变  | ✔ 允许      | 整数下标        | 动态数组, 需要频繁修改  |
| 元组<br>(tuple) | ✔ 有序           | ✘ 不可变 | ✔ 允许      | 整数下标        | 不可变的数据记录, 字典键 |
| 字典<br>(dict)  | ✔ 有序<br>(3.7+) | ✔ 可变  | 键唯一, 值可重复 | 键 (任意不可变类型) | 键值映射, 快速查找    |
| 集合<br>(set)   | ✘ 无序           | ✔ 可变  | ✘ 元素唯一    | 无           | 去重、集合运算       |

选择口诀:

- 需要频繁增删改 → 列表
- 需要保护数据不被修改 → 元组
- 需要通过名字 (键) 查找 → 字典
- 需要去重或集合运算 → 集合